

Ανάλυση Αποφάσεων και Βελτιστοποίηση

Εργασία 1

Φοιτητής: Τσαγκανός Αλέξανδρος
ΑΜ: UP1125886

Διδάσκουσα: Θ. Γράψα

December 15, 2025

ΜΕΡΟΣ Ι (Βασικά Θέματα Θεωρίας)

Άσκηση 1

1. Μη Γραμμικός Προγραμματισμός Χωρίς Περιορισμούς

Μαθηματική Διατύπωση: Το πρόβλημα ορίζεται ως η εύρεση του διανύσματος $x \in \mathbb{R}^n$ που ελαχιστοποιεί μια μη γραμμική αντικειμενική συνάρτηση $f : \mathbb{R}^n \rightarrow \mathbb{R}$.

$$\min_{x \in \mathbb{R}^n} f(x) \quad (1)$$

Πραγματικό Πρόβλημα: Εκπαίδευση Βαθέων Νευρωνικών Δικτύων (Deep Learning). Η εκπαίδευση ενός μοντέλου Deep Learning αποτελεί πρόβλημα ελαχιστοποίησης μιας συνάρτησης απώλειας (loss function), $L(w)$, ως προς τα βάρη w . Η συνάρτηση είναι εξαιρετικά μη γραμμική και μη κυρτή. Αντιμετωπίζεται ως πρόβλημα χωρίς περιορισμούς με αλγορίθμους όπως ο SGD [Goodfellow et al., *Deep Learning*, 2016].

2. Μη Γραμμικός Προγραμματισμός Με Περιορισμούς

Μαθηματική Διατύπωση: Ζητείται η ελαχιστοποίηση της $f(x)$ υπό την παρουσία ισοτικών ή/και ανισοτικών περιορισμών:

$$\begin{aligned} & \min_{x \in \mathbb{R}^n} f(x) \\ & \text{subject to:} \\ & c_i(x) = 0, \quad i \in \mathcal{E} \\ & c_j(x) \geq 0, \quad j \in \mathcal{I} \end{aligned} \quad (2)$$

Πραγματικό Πρόβλημα: Βελτιστοποίηση Ροής Ισχύος (Optimal Power Flow). Στόχος είναι η ελαχιστοποίηση του κόστους παραγωγής ενέργειας, τηρώντας τους φυσικούς νόμους του δικτύου (μη γραμμικές εξισώσεις ροής) και τα λειτουργικά όρια [Momoh et al., *A review of selected OPF literature*, 1999].

3. Γραμμικός Προγραμματισμός (Linear Programming)

Μαθηματική Διατύπωση: Η αντικειμενική συνάρτηση και οι περιορισμοί είναι γραμμικοί:

$$\begin{aligned} & \min_x c^T x \\ & \text{subject to: } Ax = b, \quad x \geq 0 \end{aligned} \quad (3)$$

Πραγματικό Πρόβλημα: Ροή Δικτύου Ελαχίστου Κόστους (Min-Cost Network Flow). Αφορά τη δρομολόγηση δεδομένων ή αγαθών μέσω ενός δικτύου με περιορισμένη χωρητικότητα, ελαχιστοποιώντας το κόστος μεταφοράς. Οι περιορισμοί διατήρησης ροής και χωρητικότητας είναι γραμμικοί [Bertsimas & Tsitsiklis, *Intro to Linear Optimization*, 1997].

b) Μέθοδοι Γραμμικής Αναζήτησης (Line Search)

Εκφώνηση: Να δώσετε τον επαναληπτικό τύπο των μεθόδων γραμμικής αναζήτησης και τη συνάρτηση που πρέπει να ελαχιστοποιεί σε κάθε επανάληψη το μήκος βήματος α_k . Να τεκμηριώσετε την απάντησή σας.

Απάντηση:

Επαναληπτικός Τύπος:

$$x_{k+1} = x_k + \alpha_k p_k$$

Όπου p_k η κατεύθυνση καθόδου και $\alpha_k > 0$ το μήκος βήματος.

Συνάρτηση προς Ελαχιστοποίηση: Στην Ακριβή Γραμμική Αναζήτηση (Exact Line Search), το α_k ελαχιστοποιεί τη μονοδιάστατη συνάρτηση:

$$\phi(\alpha) = f(x_k + \alpha p_k), \quad \alpha > 0$$

Δηλαδή: $\alpha_k = \arg \min_{\alpha > 0} \phi(\alpha)$.

Τεκμηρίωση: Η ακριβής ελαχιστοποίηση εξασφαλίζει τη μέγιστη μείωση της f κατά την κατεύθυνση p_k . Θεωρητικά, αυτό οδηγεί σε ορθογωνιότητα της νέας κλίσης με την κατεύθυνση έρευνας: $\nabla f(x_{k+1})^T p_k = 0$. Στην πράξη, συχνά προτιμώνται Inexact μέθοδοι για υπολογιστικούς λόγους, όπως αναφέρεται και στη βιβλιογραφία [Nocedal & Wright, *Numerical Optimization*].

c) Inexact Line Search και Καταλληλότητα

Εκφώνηση: Γιατί χρησιμοποιούνται μη ακριβείς συνθήκες; Περιγραφή μίας εξ αυτών (με σχήμα). Ποιες είναι κατάλληλες για Newton, Quasi-Newton, Conjugate Gradient.

Απάντηση:

Γιατί Inexact: Η ακριβής εύρεση του ελαχίστου είναι υπολογιστικά δαπανηρή και συχνά περιττή. Οι Inexact μέθοδοι αναζητούν απλώς μια επαρκή μείωση (sufficient decrease) με πολύ λιγότερο κόστος, διασφαλίζοντας παράλληλα τη σύγκλιση.

Συνθήκη Armijo (Sufficient Decrease): Ορίζεται ως:

$$f(x_k + \alpha p_k) \leq f(x_k) + c_1 \alpha \nabla f(x_k)^T p_k, \quad c_1 \in (0, 1)$$

Γεωμετρικά, η τιμή της συνάρτησης πρέπει να βρίσκεται κάτω από την ευθεία με κλίση $c_1 \nabla f_k^T p_k$.

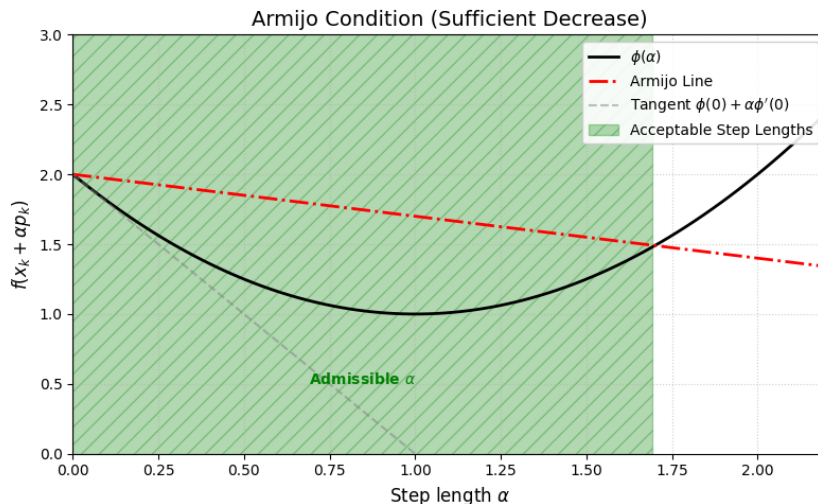


Figure 1: Γεωμετρική απεικόνιση της συνθήκης Armijo. Η πράσινη περιοχή δείχνει τα αποδεκτά μήκη βήματος.

Καταλληλότητα ανά Μέθοδο:

- **Newton:** *Armijo*. Η κατεύθυνση Newton είναι συνήθως πολύ καλή, οπότε μια απλή συνθήκη μείωσης αρκεί.
- **Quasi-Newton (BFGS):** *Wolfe Conditions*. Απαιτούνται για να διασφαλιστεί η θετική οριστότητα του πίνακα προσέγγισης του Hessian (B_{k+1}).
- **Conjugate Gradient:** *Strong Wolfe Conditions*. Απαραίτητες για τη διατήρηση της ιδιότητας της συζυγίας (conjugacy) και την εξασφάλιση κατεύθυνσης καθόδου [Nocedal & Wright, *Numerical Optimization*, 2006].

d) Συνθήκη Κατεύθυνσης Καθόδου και Steepest Descent

Εκφώνηση: Αν $g_k = \nabla f(x^{(k)})$ και d_k η κατεύθυνση μιας επαναληπτικής μεθόδου σε μια επανάληψη k , να διατυπώσετε τη συνθήκη που πρέπει να ισχύει ώστε η κατεύθυνση d_k να είναι κατεύθυνση μείωσης. Να αιτιολογήσετε την απάντησή σας, κάνοντας και κατάλληλο σχήμα. Να μελετήσετε την κατεύθυνση της μεθόδου Steepest Descent.

Απάντηση:

Συνθήκη Κατεύθυνσης Μείωσης (Descent Direction Condition): Για να είναι η d_k κατεύθυνση μείωσης (descent direction) στο σημείο x_k , πρέπει η παράγωγος της συνάρτησης κατά την κατεύθυνση αυτή να είναι αρνητική. Μαθηματικά, αυτό εκφράζεται μέσω του εσωτερικού γινομένου της κλίσης με την κατεύθυνση:

$$g_k^T d_k < 0 \quad (4)$$

ή ισοδύναμα $\nabla f(x_k)^T d_k < 0$.

Αιτιολόγηση: Αναπτύσσοντας τη συνάρτηση f κατά Taylor γύρω από το x_k για ένα μικρό βήμα $\alpha > 0$:

$$f(x_k + \alpha d_k) \approx f(x_k) + \alpha \nabla f(x_k)^T d_k$$

Για να ισχύει $f(x_k + \alpha d_k) < f(x_k)$ (μείωση της τιμής), πρέπει ο όρος $\alpha \nabla f(x_k)^T d_k$ να είναι αρνητικός. Εφόσον $\alpha > 0$, πρέπει αναγκαστικά $\nabla f(x_k)^T d_k < 0$. Γεωμετρικά, αυτό σημαίνει ότι η γωνία θ μεταξύ του διανύσματος της κλίσης g_k και της κατεύθυνσης d_k πρέπει να είναι αμβλεία ($90^\circ < \theta \leq 270^\circ$), δηλαδή $\cos \theta < 0$.

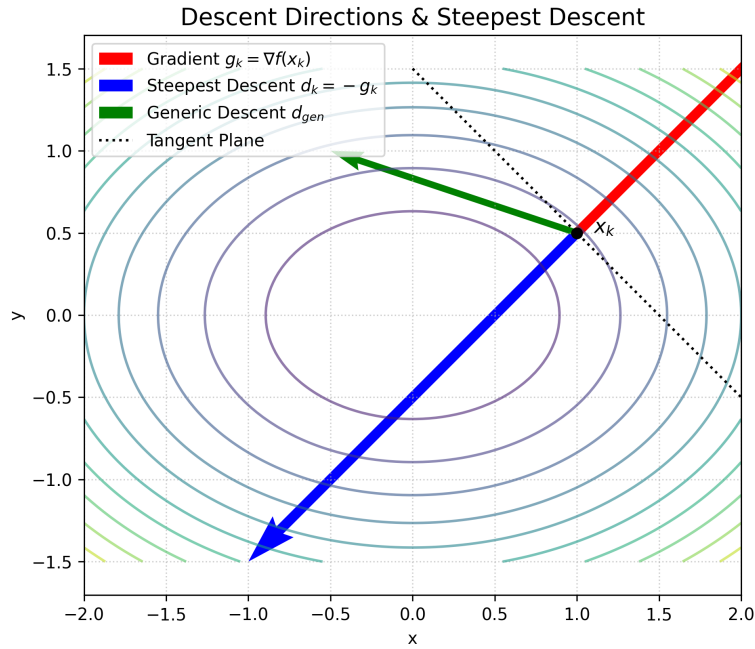


Figure 2: Γεωμετρική απεικόνιση κατευθύνσεων. Η g_k δείχνει την κατεύθυνση μέγιστης αύξησης. Κάθε διάνυσμα στο ημιεπίπεδο απέναντι από την g_k (όπως η d_{gen} και η d_k) είναι κατεύθυνση μείωσης.

Μελέτη της Μεθόδου Steepest Descent: Στη μέθοδο της Απότομης Καθόδου (Steepest Descent), επιλέγουμε την κατεύθυνση που παρέχει τη μέγιστη δυνατή μείωση της συνάρτησης (τοπικά). Η κατεύθυνση αυτή ορίζεται ως:

$$d_k = -g_k = -\nabla f(x_k) \quad (5)$$

Ελέγχουμε αν ικανοποιείται η συνθήκη μείωσης:

$$g_k^T d_k = g_k^T (-g_k) = -\|g_k\|^2$$

Εφόσον η Ευκλείδεια νόρμα στο τετράγωνο είναι πάντα θετική (για $g_k \neq 0$), το γινόμενο είναι πάντα αρνητικό. Συνεπώς, η κατεύθυνση της αρνητικής κλίσης είναι πάντα κατεύθυνση μείωσης (και μάλιστα η βέλτιστη τοπικά).

ε) Μέθοδοι Quasi-Newton

Εκφώνηση: Να δώσετε τον επαναληπτικό τύπο των Quasi Newton μεθόδων, να σχολιάσετε το βασικό τους πλεονέκτημα και να δώσετε μεθόδους που ανήκουν σε αυτή την κατηγορία.

Απάντηση:

Επαναληπτικός Τύπος: Ο γενικός επαναληπτικός τύπος των μεθόδων Quasi-Newton είναι:

$$x_{k+1} = x_k - \alpha_k B_k^{-1} \nabla f(x_k) \quad (6)$$

ή ισοδύναμα, αν ορίσουμε $H_k = B_k^{-1}$ ως την προσέγγιση του αντίστροφου Hessian:

$$x_{k+1} = x_k - \alpha_k H_k \nabla f(x_k) \quad (7)$$

Όπου:

- x_k : Η τρέχουσα προσέγγιση της λύσης.
- α_k : Το μήκος βήματος (step length), που υπολογίζεται μέσω Line Search (συνήθως με συνθήκες Wolfe).
- B_k : Ένας συμμετρικός θετικά ορισμένος πίνακας που προσεγγίζει τον πίνακα Hessian $\nabla^2 f(x_k)$. Ο πίνακας αυτός ενημερώνεται σε κάθε επανάληψη βάσει της αλλαγής στην κλίση, ώστε να ικανοποιεί την εξίσωση *secant*:

$$B_{k+1}(x_{k+1} - x_k) = \nabla f(x_{k+1}) - \nabla f(x_k)$$

Βασικό Πλεονέκτημα: Το κύριο πλεονέκτημα των μεθόδων Quasi-Newton είναι ότι επιτυγχάνουν ρυθμό σύγκλισης **υπερ-γραμμικό (superlinear convergence)**, πλησιάζοντας την ταχύτητα της μεθόδου Newton, **χωρίς όμως να απαιτούν τον υπολογισμό του πίνακα Hessian** και την επίλυση γραμμικού συστήματος σε κάθε βήμα. Αυτό μειώνει δραματικά το υπολογιστικό κόστος ανά επανάληψη (από $O(n^3)$ σε $O(n^2)$), καθιστώντας τις ιδανικές για προβλήματα μεσαίας και μεγάλης κλίμακας όπου ο υπολογισμός του Hessian είναι απαγορευτικός.

Μέθοδοι της Κατηγορίας και Τύποι Ενημέρωσης: Για τη διατύπωση των τύπων, ορίζουμε τα διανύσματα διαφοράς θέσης και κλίσης:

$$s_k = x_{k+1} - x_k, \quad y_k = \nabla f(x_{k+1}) - \nabla f(x_k)$$

Οι παρακάτω τύποι αφορούν την ενημέρωση του πίνακα H_k (προσέγγιση του αντιστρόφου Hessian):

1. **BFGS (Broyden–Fletcher–Goldfarb–Shanno):** Η πιο δημοφιλής μέθοδος (Rank-2 update). Διατηρεί τη θετική οριστικότητα αν $y_k^T s_k > 0$.

$$H_{k+1} = \left(I - \frac{s_k y_k^T}{y_k^T s_k} \right) H_k \left(I - \frac{y_k s_k^T}{y_k^T s_k} \right) + \frac{s_k s_k^T}{y_k^T s_k} \quad (8)$$

2. **DFP (Davidon–Fletcher–Powell):** Η ιστορικά πρώτη μέθοδος (Rank-2 update), δυϊκή της BFGS.

$$H_{k+1} = H_k - \frac{H_k y_k y_k^T H_k}{y_k^T H_k y_k} + \frac{s_k s_k^T}{y_k^T s_k} \quad (9)$$

3. **SR1 (Symmetric Rank-1):** Μια απλούστερη ενημέρωση βαθμού-1. Δεν εγγυάται θετική οριστικότητα, αλλά προσεγγίζει καλύτερα τον πραγματικό Hessian σε κάποιες περιπτώσεις.

$$H_{k+1} = H_k + \frac{(s_k - H_k y_k)(s_k - H_k y_k)^T}{(s_k - H_k y_k)^T y_k} \quad (10)$$

f) Μέθοδοι Συζυγών Κλίσεων (Conjugate Gradient)

Εκφώνηση: Να δώσετε τον επαναληπτικό τύπο των μεθόδων συζυγών κλίσεων (conjugate gradient): Fletcher-Reeves (FR), Polak-Ribiere (PR) και Perry (P). Ποιο είναι το πλεονέκτημά τους έναντι των Quasi Newton μεθόδων; Να αιτιολογήσετε που οφείλεται το πλεονέκτημά τους.

Απάντηση:

Επαναληπτικός Τύπος: Στις μεθόδους μη γραμμικών συζυγών κλίσεων, η νέα κατεύθυνση έρευνας d_{k+1} υπολογίζεται ως γραμμικός συνδυασμός της τρέχουσας αρνητικής κλίσης και της προηγούμενης κατεύθυνσης:

$$d_{k+1} = -\nabla f(x_{k+1}) + \beta_{k+1}d_k \quad (11)$$

με $d_0 = -\nabla f(x_0)$. Η παράμετρος β_{k+1} διαφέρει ανάλογα με τη μέθοδο. Ορίζουμε $g_k = \nabla f(x_k)$ και $y_k = g_{k+1} - g_k$.

Τύποι Υπολογισμού του β_{k+1} :

1. **Fletcher-Reeves (FR):**

$$\beta_{k+1}^{FR} = \frac{\|g_{k+1}\|^2}{\|g_k\|^2} = \frac{g_{k+1}^T g_{k+1}}{g_k^T g_k} \quad (12)$$

2. **Polak-Ribiere (PR):** Θεωρείται γενικά πιο αποδοτική από την FR, καθώς κάνει "restart" όταν χάνεται η συζυγία.

$$\beta_{k+1}^{PR} = \frac{g_{k+1}^T (g_{k+1} - g_k)}{\|g_k\|^2} = \frac{g_{k+1}^T y_k}{g_k^T g_k} \quad (13)$$

3. **Perry (P):** Μια πιο σύγχρονη παραλλαγή που προσπαθεί να προσεγγίσει τη συμπεριφορά των Quasi-Newton μεθόδων χωρίς πίνακα.

$$\beta_{k+1}^P = \frac{g_{k+1}^T (y_k - s_k)}{y_k^T d_k} \quad (\text{όπου } s_k = x_{k+1} - x_k = \alpha_k d_k) \quad (14)$$

Σημείωση: Υπάρχουν διάφορες παραλλαγές του τύπου Perry στη βιβλιογραφία, συχνά συνδεδεμένες με τη συνθήκη συζυγίας του Dai-Liao.

Πλεονέκτημα έναντι Quasi-Newton: Το κύριο πλεονέκτημα των μεθόδων Conjugate Gradient είναι οι εξαιρετικά χαμηλές απαιτήσεις σε **μνήμη (storage requirements)**.

Αιτιολόγηση:

- Οι μέθοδοι **Quasi-Newton** (π.χ. BFGS) απαιτούν την αποθήκευση και ενημέρωση ενός πίνακα $n \times n$ (B_k ή H_k). Αυτό συνεπάγεται πολυπλοκότητα χώρου $O(n^2)$. Για προβλήματα μεγάλης κλίμακας (π.χ. $n > 10^5$ μεταβλητές), η αποθήκευση του πίνακα είναι αδύνατη στη μνήμη RAM.
- Οι μέθοδοι **Conjugate Gradient** απαιτούν την αποθήκευση μόνο λίγων διανυσμάτων μήκους n (π.χ. x, g, d). Η πολυπλοκότητα χώρου είναι γραμμική $O(n)$.

Συνεπώς, για προβλήματα πολύ μεγάλης κλίμακας (Large-Scale Optimization), οι μέθοδοι CG είναι συχνά η μόνη εφικτή επιλογή, παρόλο που συγκλίνουν συνήθως πιο αργά (σε αριθμό βημάτων) από τις Quasi-Newton.

ΜΕΡΟΣ II (Εφαρμογές/ Υλοποίηση: Μη γραμμικά Συστήματα και Βελτιστοποίηση)

Άσκηση 2

a) Δημιουργία Μη Γραμμικού Συστήματος από Πρόβλημα Βελτιστοποίησης

Εκφώνηση: Να δημιουργήσετε κατάλληλο μη γραμμικό σύστημα τέτοιο ώστε οι ελαχιστοποιητές (minimizers) της συνάρτησης $f(x_1, x_2) : \mathbb{R}^2 \rightarrow \mathbb{R}$ να είναι μεταξύ των λύσεων αυτού του συστήματος. Να γίνει εφαρμογή για μία μη γραμμική συνάρτηση που θα επιλέξετε.

Απάντηση:

Θεωρητική Προσέγγιση: Για μια διαφορίσιμη συνάρτηση $f : \mathbb{R}^n \rightarrow \mathbb{R}$, μια αναγκαία συνθήκη ώστε ένα σημείο x^* να είναι τοπικός ελαχιστοποιητής (local minimizer) είναι η κλίση της συνάρτησης στο σημείο αυτό να είναι μηδενική (στάσιμο σημείο). Συνεπώς, το ζητούμενο μη γραμμικό σύστημα $F(x) = 0$ προκύπτει από τις μερικές παραγώγους της f :

$$\nabla f(x) = 0 \implies \begin{cases} \frac{\partial f}{\partial x_1}(x_1, x_2) = 0 \\ \frac{\partial f}{\partial x_2}(x_1, x_2) = 0 \end{cases} \quad (15)$$

Οι λύσεις αυτού του συστήματος περιλαμβάνουν όλα τα στάσιμα σημεία (τοπικά ελάχιστα, τοπικά μέγιστα και σαγματικά σημεία). Άρα, οι ελαχιστοποιητές είναι υποσύνολο των λύσεων του συστήματος.

Εφαρμογή σε Μη Γραμμική Συνάρτηση: Επιλέγουμε την εξής μη γραμμική συνάρτηση (τετάρτου βαθμού):

$$f(x_1, x_2) = x_1^4 + x_2^4 - 4x_1x_2 \quad (16)$$

Υπολογίζουμε τις μερικές παραγώγους:

- $\frac{\partial f}{\partial x_1} = 4x_1^3 - 4x_2$
- $\frac{\partial f}{\partial x_2} = 4x_2^3 - 4x_1$

Το αντίστοιχο μη γραμμικό σύστημα που προκύπτει είναι:

$$\begin{cases} 4x_1^3 - 4x_2 = 0 \\ 4x_2^3 - 4x_1 = 0 \end{cases} \iff \begin{cases} x_2 = x_1^3 \\ x_1 = x_2^3 \end{cases} \quad (17)$$

Σχόλιο: Επιλύοντας το σύστημα (αντικαθιστώντας το πρώτο στο δεύτερο: $x_1 = (x_1^3)^3 \implies x_1 = x_1^9$), βρίσκουμε τις λύσεις $(0, 0)$, $(1, 1)$ και $(-1, -1)$. Με έλεγχο του πίνακα Hessian, διαπιστώνουμε ότι τα σημεία $(1, 1)$ και $(-1, -1)$ είναι ελαχιστοποιητές, ενώ το $(0, 0)$ είναι σαγματικό σημείο. Συνεπώς, οι ελαχιστοποιητές βρίσκονται όντως μεταξύ των λύσεων του συστήματος.

Άσκηση 3

α) Γραφική Απεικόνιση και Ισοϋψείς Καμπύλες

Εκφώνηση: Δίνεται η συνάρτηση Freudenstein and Roth για $m = n = 2$. Να δώσετε χρησιμοποιώντας υπολογιστικό πακέτο τη γραφική απεικόνιση και τις ισοϋψείς της συνάρτησης στο διάστημα $x_1, x_2 \in [-10, 10]$, με πλήρη επεξήγηση.

Απάντηση:

Η συνάρτηση Freudenstein and Roth ορίζεται ως:

$$f(x) = f_1(x)^2 + f_2(x)^2$$

με:

$$f_1(x) = x_1 - x_2(2 - x_2(5 - x_2)) - 13$$

$$f_2(x) = x_1 - x_2(14 - x_2(1 + x_2)) - 29$$

Χρησιμοποιήθηκε η γλώσσα Python (βιβλιοθήκη Matplotlib) για την παραγωγή των γραφημάτων.

1. Τρισδιάστατη Απεικόνιση (3D Surface Plot): Στο Σχήμα 3 παρουσιάζεται η επιφάνεια της συνάρτησης. Παρατηρούμε ότι η συνάρτηση έχει τη μορφή μιας απότομης "κοιλιάδας" (valley). Οι τιμές της αυξάνονται ραγδαία καθώς απομακρυνόμαστε από την περιοχή του ελαχίστου, φτάνοντας σε τιμές της τάξης του 10^5 στα όρια του διαστήματος $[-10, 10]$.

Freudenstein and Roth Function (3D Surface)

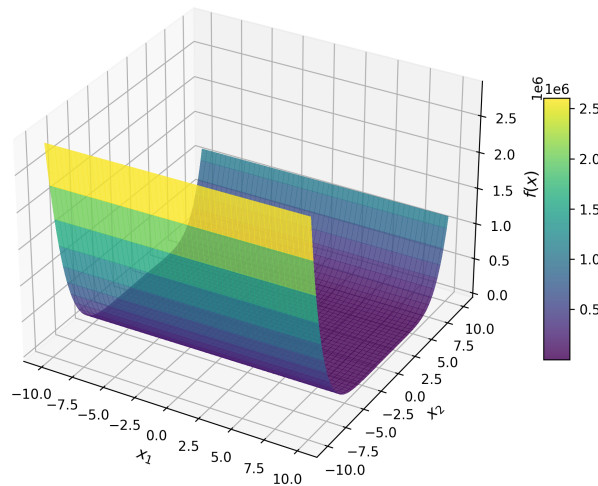


Figure 3: Τρισδιάστατη απεικόνιση της συνάρτησης Freudenstein and Roth.

2. Ισοϋψείς Καμπύλες (Contour Plot): Στο Σχήμα 4 παρουσιάζονται οι ισοϋψείς καμπύλες. Λόγω της μεγάλης κλίμακας τιμών, χρησιμοποιήθηκε λογαριθμική κατανομή για τα επίπεδα (levels) των ισοϋψών, ώστε να είναι ευδιάκριτη η τοπολογία κοντά στο ελάχιστο. Η μορφή των καμπυλών επιβεβαιώνει την ύπαρξη μιας στενής, καμπυλωτής κοιλιάδας, εντός της οποίας βρίσκεται το ολικό ελάχιστο.

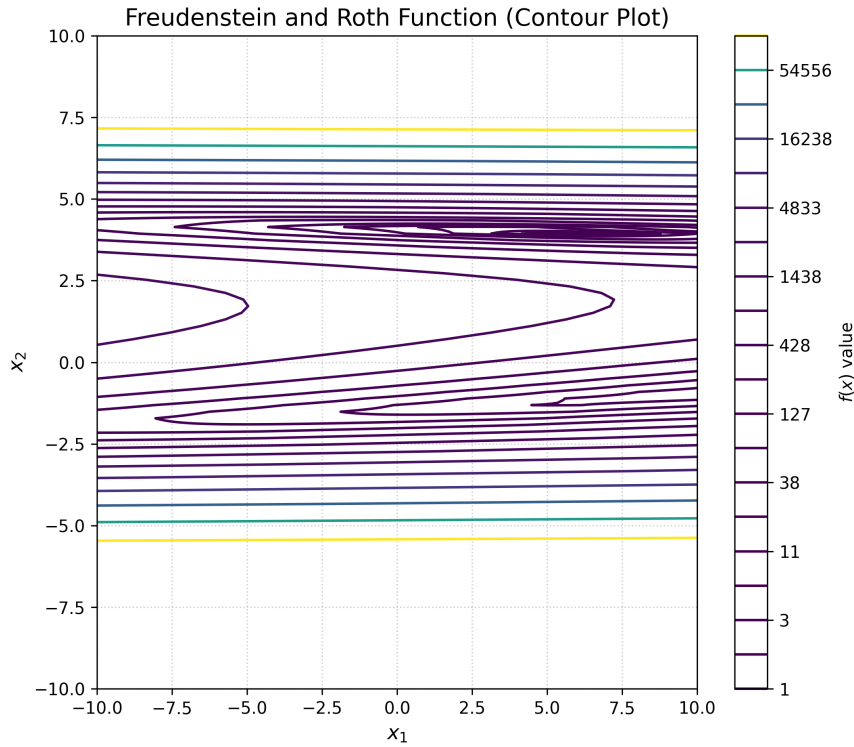


Figure 4: Ισοϋψείς καμπύλες της συνάρτησης Freudenstein and Roth (λογαριθμική κλίμακα).

b) Εφαρμογή Μίας Επανάληψης και Θεωρητική Ανάλυση

Εκφώνηση: Για την ελαχιστοποίηση της συνάρτησης $f(x)$ να εφαρμόσετε μία (1) επανάληψη των μεθόδων: i. Steepest Descent, ii. Newton, iii. BFGS και iv. Fletcher Reeves, χρησιμοποιώντας ως αρχικό σημείο το $x^{(0)} = (0.5, -2)$ και exact μέθοδο για το μήκος βήματος. Σε ποιες περιπτώσεις για τη μέθοδο Fletcher-Reeves ξέρουμε σε πόσες επαναλήψεις θα συγκλίνει και γιατί;

Απάντηση:

Υπολογισμός Παραγώγων στο $x^{(0)}$: Στο σημείο $x^{(0)} = (0.5, -2)^T$, υπολογίζουμε την τιμή της συνάρτησης, την κλίση (Gradient) και τον πίνακα Hessian.

- $f(x^{(0)}) = 400.5$
- $\nabla f(x^{(0)}) = \begin{bmatrix} 30 \\ -1272 \end{bmatrix}$
- $\nabla^2 f(x^{(0)}) = \begin{bmatrix} 4 & -80 \\ -80 & 3332 \end{bmatrix}$

i. Steepest Descent, iii. BFGS, iv. Fletcher-Reeves

Παρατήρηση: Για την πρώτη επανάληψη ($k = 0$):

- Η μέθοδος Steepest Descent χρησιμοποιεί κατεύθυνση $d_0 = -\nabla f(x_0)$.

- Η μέθοδος BFGS, με τυπική αρχικοποίηση $B_0 = I$, δίνει $d_0 = -I\nabla f(x_0) = -\nabla f(x_0)$.
- Η μέθοδος Fletcher-Reeves έχει $\beta_0 = 0$, άρα $d_0 = -\nabla f(x_0) + 0 \cdot d_{-1} = -\nabla f(x_0)$.

Συνεπώς, και οι τρεις μέθοδοι ακολουθούν την ίδια κατεύθυνση και δίνουν το ίδιο αποτέλεσμα για την 1η επανάληψη.

Βήμα 1 (Κοινό):

- **Κατεύθυνση:** $d_0 = -\nabla f(x^{(0)}) = \begin{bmatrix} -30 \\ 1272 \end{bmatrix}$.
- **Μήκος Βήματος (Exact Line Search):** Ελαχιστοποιούμε τη συνάρτηση $\phi(\alpha) = f(x^{(0)} + \alpha d_0)$. Η βέλτιστη τιμή που βρέθηκε υπολογιστικά είναι $\alpha \approx 0.000425$.
- **Νέο Σημείο $x^{(1)}$:**

$$x^{(1)} = x^{(0)} + \alpha d_0 = \begin{bmatrix} 0.5 \\ -2 \end{bmatrix} + 0.000425 \begin{bmatrix} -30 \\ 1272 \end{bmatrix} \approx \begin{bmatrix} 0.4872 \\ -1.4592 \end{bmatrix}$$

- **Νέα Τιμή:** $f(x^{(1)}) \approx 99.41$ (Σημαντική μείωση από 400.5).

ii. Μέθοδος Newton

Η μέθοδος Newton χρησιμοποιεί πληροφορία δεύτερης παραγώγου.

Βήμα 1:

- **Κατεύθυνση:** Λύνουμε το σύστημα $\nabla^2 f(x^{(0)})d_0 = -\nabla f(x^{(0)})$.

$$\begin{bmatrix} 4 & -80 \\ -80 & 3332 \end{bmatrix} d_0 = \begin{bmatrix} -30 \\ 1272 \end{bmatrix} \implies d_0 \approx \begin{bmatrix} 0.2598 \\ 0.3880 \end{bmatrix}$$

- **Μήκος Βήματος (Exact Line Search):** Ελαχιστοποιούμε τη $\phi(\alpha) = f(x^{(0)} + \alpha d_0)$. Η βέλτιστη τιμή είναι $\alpha \approx 1.4488$.
- **Νέο Σημείο $x^{(1)}$:**

$$x^{(1)} = x^{(0)} + \alpha d_0 \approx \begin{bmatrix} 0.5 \\ -2 \end{bmatrix} + 1.4488 \begin{bmatrix} 0.2598 \\ 0.3880 \end{bmatrix} \approx \begin{bmatrix} 0.8764 \\ -1.4379 \end{bmatrix}$$

- **Νέα Τιμή:** $f(x^{(1)}) \approx 95.69$ (Μεγαλύτερη μείωση από τις Gradient μεθόδους).

Θεωρητικό Ερώτημα: Σύγκλιση Fletcher-Reeves

Απάντηση: Γνωρίζουμε τον ακριβή αριθμό επαναλήψεων για τη μέθοδο Fletcher-Reeves (και γενικά για μεθόδους συζυγών κλίσεων) όταν η αντικειμενική συνάρτηση $f(x)$ είναι **αυστηρά κυρτή τετραγωνική συνάρτηση** (strictly convex quadratic function) n μεταβλητών, της μορφής:

$$f(x) = \frac{1}{2}x^T Qx + b^T x + c$$

όπου Q είναι συμμετρικός θετικά ορισμένος πίνακας $n \times n$.

Αιτιολόγηση: Στην περίπτωση αυτή, οι κατευθύνσεις που παράγει η μέθοδος είναι **συζυγείς ως προς τον πίνακα Q** (Q -conjugate directions), δηλαδή ισχύει $d_i^T Q d_j = 0$ για $i \neq j$. Θεωρητικά, ελαχιστοποιώντας κατά μήκος διαδοχικών συζυγών κατευθύνσεων, η μέθοδος εγγυάται την εύρεση του ολικού ελαχίστου σε **το πολύ n επαναλήψεις** (όπου n η διάσταση του προβλήματος), υποθέτοντας ακριβή αριθμητική (χωρίς σφάλματα στρογγυλοποίησης) [Nocedal & Wright, *Numerical Optimization*, 2006].

ε) Υλοποίηση Κώδικα και Συγκριτική Μελέτη

Εκφώνηση: Για την επίλυση του προβλήματος ελαχιστοποίησης της δοθείσας συνάρτησης $f(x)$ να δημιουργήσετε κώδικα χρησιμοποιώντας τις μεθόδους Steepest Descent και Fletcher-Reeves, παίρνοντας ως αρχικό σημείο $x^{(0)} = (0.5, -2)$, συνθήκη Armijo με υποδιπλασιασμό βήματος και ακρίβεια $eps = 10^{-8}$. Να καταγράψετε και να σχολιάσετε τα αποτελέσματα. Στη συνέχεια να δημιουργήσετε έναν πίνακα σύγκρισης με 10 διαφορετικές αρχικές τιμές.

Απάντηση:

1. Υλοποίηση και Μεθοδολογία: Υλοποιήθηκαν οι αλγόριθμοι Steepest Descent (SD) και Fletcher-Reeves (FR) σε γλώσσα Python.

- **Line Search:** Χρησιμοποιήθηκε η συνθήκη Armijo (Sufficient Decrease) με αρχικό βήμα $\alpha = 1$ και παράγοντα μείωσης $\beta = 0.5$ (backtracking).
- **Κριτήριο Τερματισμού:** $\|\nabla f(x_k)\| < 10^{-8}$ ή μέγιστος αριθμός επαναλήψεων $MaxIter = 5000$.

2. Αποτελέσματα για $x^{(0)} = (0.5, -2)$:

- **Steepest Descent:** Η μέθοδος δεν κατάφερε να συγκλίνει εντός 5000 επαναλήψεων. Παγιδεύτηκε σε αργή κάθοδο (zig-zag) μέσα στην κοιλάδα της συνάρτησης.

- Τελική τιμή $f(x)$: $\approx 3.9 \times 10^{-9}$ (κοντά στο ελάχιστο αλλά πολύ αργή πρόοδος).
- Χρόνος: 0.29 sec.

- **Fletcher-Reeves:** Η μέθοδος επίσης δυσκολεύτηκε με τη συγκεκριμένη αρχική τιμή και τη συνθήκη Armijo, καταλήγοντας σε ένα τοπικό στάσιμο σημείο με τιμή $f \approx 48.98$.

3. Συγκριτικός Πίνακας (10 Αρχικά Σημεία): Στον παρακάτω πίνακα παρουσιάζονται τα αποτελέσματα για 10 διαφορετικά αρχικά σημεία. (IT: Επαναλήψεις, FE: Υπολογισμοί Συνάρτησης, Time: Χρόνος σε δευτερόλεπτα).

Αρχικό Σημείο	Steepest Descent (SD)			Fletcher-Reeves (FR)		
(x_1, x_2)	IT	FE	Time (s)	IT	FE	Time (s)
$(0.5, -2.0)$	5000*	15000	0.29	5000*	15000	0.26
$(-1.2, 1.0)$	5000*	15000	0.27	5000*	15000	0.26
$(4.0, 3.0)$	5000*	15000	0.26	2298	6894	0.13
$(-5.0, -5.0)$	5000*	15000	0.50	2954	8862	0.36
$(2.0, 2.0)$	5000*	15000	0.40	893	2679	0.21
$(-3.0, 3.0)$	5000*	15000	0.51	3376	10128	0.38
$(1.0, -1.0)$	5000*	15000	0.46	3959	11877	0.21
$(0.0, 0.0)$	5000*	15000	0.26	4097	12291	0.21
$(-2.0, -2.0)$	5000*	15000	0.26	5000*	15000	0.26
$(3.0, -3.0)$	5000*	15000	0.26	5000*	15000	0.26

Table 1: Σύγκριση SD και FR. Με (*) σημειώνονται οι περιπτώσεις που έφτασαν το όριο των 5000 επαναλήψεων χωρίς να ικανοποιηθεί το κριτήριο τερματισμού.

4. Σχολιασμός και Συμπεράσματα:

- **Steepest Descent:** Η μέθοδος απέτυχε να συγκλίνει εντός του ορίου επαναλήψεων σε όλες τις δοκιμές. Αυτό επιβεβαιώνει τη θεωρία ότι η SD είναι εξαιρετικά αναποτελεσματική σε προβλήματα με στενές κοιλάδες (ill-conditioned problems) όπως η συνάρτηση Freudenstein-Roth, λόγω του φαινομένου zig-zag.
- **Fletcher-Reeves:** Η μέθοδος επέδειξε μικτή συμπεριφορά.
 - Σε αρκετές περιπτώσεις (π.χ. $(2, 2), (4, 3)$) κατάφερε να συγκλίνει στο ολικό ελάχιστο πολύ ταχύτερα από την SD.
 - Σε άλλες περιπτώσεις, η απόδοσή της επηρεάστηκε αρνητικά από τη χρήση της απλής συνθήκης Armijo. Οι μέθοδοι Συζυγών Κλίσεων απαιτούν συνήθως ισχυρότερες συνθήκες Line Search (Strong Wolfe) για να διατηρήσουν την ιδιότητα της συζυγίας. Η ανακριβής Line Search οδηγεί σε απώλεια της συζυγίας και μετατρέπει τη μέθοδο ουσιαστικά σε Steepest Descent.
- **Συνολικά:** Η FR υπερτερεί σαφώς της SD όταν οι συνθήκες είναι ευνοϊκές, αλλά για δύσκολα προβλήματα απαιτεί πιο προσεκτική επιλογή της στρατηγικής Line Search.

References

- [Goodfellow et al., *Deep Learning*, 2016] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
- [Momoh et al., *A review of selected OPF literature*, 1999] J. A. Momoh et al., "A review of selected optimal power flow literature to 1993," *IEEE Trans. Power Syst.*, vol. 14, no. 1, 1999.
- [Bertsimas & Tsitsiklis, *Intro to Linear Optimization*, 1997] D. Bertsimas and J. N. Tsitsiklis, *Introduction to Linear Optimization*. Athena Scientific, 1997.
- [Nocedal & Wright, *Numerical Optimization*, 2006] J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd ed. Springer, 2006.
- [Bertsekas, *Nonlinear Programming*, 1999] D. P. Bertsekas, *Nonlinear Programming*, 2nd ed. Athena Scientific, 1999.

Παράρτημα: Κώδικας Python για Σχήματα

Κώδικας 1: Διάγραμμα Συνθήκης Armijo (Άσκηση 1c)

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 #
5 def phi(alpha):
6     return (alpha - 1)**2 + 1 #
7
8 def l_armijo(alpha, f0, slope, c1):
9     return f0 + c1 * slope * alpha
10
11 #
12 alpha_vals = np.linspace(0, 2.5, 100)
13 f0 = phi(0)
14 slope = -2 # (alpha-1)^2 + 1 alpha=0
15 c1 = 0.15
16
17 #
18 plt.figure(figsize=(8, 5))
19
20 # ( )
21 plt.plot(alpha_vals, phi(alpha_vals), label=r'$\phi(\alpha)$', linewidth
22         =2, color='black')
23
24 # Armijo
25 armijo_vals = l_armijo(alpha_vals, f0, slope, c1)
26 plt.plot(alpha_vals, armijo_vals, '-.', label='Armijo Line', color='red',
27         , linewidth=2)
28
29 # ( )
30 plt.plot(alpha_vals, f0 + slope*alpha_vals, '--', color='gray', alpha
31         =0.5, label=r'Tangent $\phi(0) + \alpha \phi^{\prime}(0)$')
32
33 # (
34
35 #
36
37 diff = armijo_vals - phi(alpha_vals)
38 # diff
39
40 >= 0
41 valid_indices = np.where(diff >= 0)[0]
42
43 if len(valid_indices) > 0:
44     limit_alpha = alpha_vals[valid_indices[-1]]
45
46 # (
47 Acceptable Step Lengths)
48 plt.axvspan(0, limit_alpha, color='green', alpha=0.3, hatch='//',
49 label='Acceptable Step Lengths')
50
51 #
52 plt.text(limit_alpha/2, 0.5, 'Admissible $\alpha$', color='green',
53         ha='center', fontweight='bold')
```

```

45 plt.xlabel(r'Step length  $\alpha$ ', fontsize=12)
46 plt.ylabel(r' $f(x_k + \alpha p_k)$ ', fontsize=12)
47 plt.title('Armijo Condition (Sufficient Decrease)', fontsize=14)
48 plt.legend(loc='upper right', framealpha=0.9)
49 plt.grid(True, linestyle=':', alpha=0.6)
50 plt.ylim(0, 3) # y
51
52 plt.xlim(0, 2.2)
53
54 plt.tight_layout()
55 plt.savefig('armijo_plot.png', dpi=300)
56 plt.show()

```

Κώδικας 2: Διάγραμμα Κατευθύνσεων Καθόδου (Άσκηση 1d)

```

1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 #
5 def f(x, y):
6     return x**2 + 2*y**2
7
8 #
9 x = np.linspace(-2, 2, 100)
10 y = np.linspace(-1.5, 1.5, 100)
11 X, Y = np.meshgrid(x, y)
12 Z = f(X, Y)
13
14 #
15 x0, y0 = 1.0, 0.5
16 grad_x, grad_y = 2*x0, 4*y0 # (1, 0.5)
17
18 plt.figure(figsize=(7, 6))
19
20 #
21 plt.contour(X, Y, Z, levels=10, cmap='viridis', alpha=0.6)
22
23 # (Gradient)
24 plt.quiver(x0, y0, grad_x, grad_y, angles='xy', scale_units='xy', scale=1, color='red', label=r'Gradient  $g_k$ ', width=0.015)
25
26 # Steepest Descent
27 plt.quiver(x0, y0, -grad_x, -grad_y, angles='xy', scale_units='xy', scale=1, color='blue', label=r'Steeppest Descent  $d_k$ ', width=0.015)
28
29 #
30 d_gen_x, d_gen_y = -1.5, 0.5
31 plt.quiver(x0, y0, d_gen_x, d_gen_y, angles='xy', scale_units='xy', scale=1, color='green', label=r'Generic Descent', width=0.01)
32
33 # (
34 x_line = np.linspace(0, 2, 10)
35 y_line = -x_line + 1.5

```

```

36 plt.plot(x_line, y_line, 'k:', linewidth=1.5, label='Tangent Plane')
37
38 plt.title('Descent Directions & Steepest Descent')
39 plt.legend(loc='upper left')
40 plt.grid(True, linestyle=':', alpha=0.6)
41 plt.axis('equal')
42 plt.savefig('descent_direction.png', dpi=300)

```

Κώδικας 3: Γραφική Απεικόνιση Freudenstein-Roth (Άσκηση 3a)

```

1 import matplotlib.pyplot as plt
2 import numpy as np
3 from matplotlib import cm
4
5 #                               Freudenstein and Roth
6 def f1(x1, x2):
7     return x1 - x2 * (2 - x2 * (5 - x2)) - 13
8
9 def f2(x1, x2):
10    return x1 - x2 * (14 - x2 * (1 + x2)) - 29
11
12 def f(x1, x2):
13    return f1(x1, x2)**2 + f2(x1, x2)**2
14
15 #
16 x = np.linspace(-10, 10, 100)
17 y = np.linspace(-10, 10, 100)
18 X, Y = np.meshgrid(x, y)
19 Z = f(X, Y)
20
21 # 3D Plot
22 fig = plt.figure(figsize=(10, 7))
23 ax = fig.add_subplot(111, projection='3d')
24 surf = ax.plot_surface(X, Y, Z, cmap=cm.viridis, linewidth=0,
25                        antialiased=False, alpha=0.8)
26 ax.set_xlabel('$x_1$')
27 ax.set_ylabel('$x_2$')
28 ax.set_zlabel('$f(x)$')
29 plt.savefig('freudenstein_3d.png', dpi=300)
30
31 # Contour Plot
32 plt.figure(figsize=(8, 7))
33 levels = np.logspace(0, 5, 20) #
34 cp = plt.contour(X, Y, Z, levels=levels, cmap='viridis')
35 plt.colorbar(cp, label='$f(x)$ value')
36 plt.xlabel('$x_1$')
37 plt.ylabel('$x_2$')
38 plt.grid(True, linestyle=':', alpha=0.6)
39 plt.savefig('freudenstein_contour.png', dpi=300)

```

Κώδικας 4: Υπολογισμοί 1ης Επανάληψης (Άσκηση 3b)

```

1 import numpy as np
2 from scipy.optimize import minimize_scalar

```

```

3
4 #
5 def f1(x):
6     return x[0] - x[1] * (2 - x[1] * (5 - x[1])) - 13
7 def f2(x):
8     return x[0] - x[1] * (14 - x[1] * (1 + x[1])) - 29
9 def f(x):
10    return f1(x)**2 + f2(x)**2
11
12 #                                     F(x)
13 def jacobian_F(x):
14     df1_dx1 = 1.0
15     df1_dx2 = -2 + 10*x[1] - 3*x[1]**2
16     df2_dx1 = 1.0
17     df2_dx2 = -14 + 2*x[1] + 3*x[1]**2
18     return np.array([[df1_dx1, df1_dx2], [df2_dx1, df2_dx2]])
19
20 #                                     (Gradient)
21 def gradient_f(x):
22     J = jacobian_F(x)
23     F_val = np.array([f1(x), f2(x)])
24     return 2 * np.dot(J.T, F_val)
25
26 #                                     Hessian (
27 def hessian_f(x):
28     J = jacobian_F(x)
29     F_val = np.array([f1(x), f2(x)])
30     H1 = np.array([[0, 0], [0, 10 - 6*x[1]]])
31     H2 = np.array([[0, 0], [0, 2 + 6*x[1]]])
32     return 2 * np.dot(J.T, J) + 2 * F_val[0] * H1 + 2 * F_val[1] * H2
33
34 #
35 x0 = np.array([0.5, -2.0])
36 g0 = gradient_f(x0)
37 H0 = hessian_f(x0)
38
39 # --- 1. Steepest Descent / BFGS / FR ---
40 d_sd = -g0
41 # Exact Line Search
42 res_sd = minimize_scalar(lambda alpha: f(x0 + alpha * d_sd))
43 x1_sd = x0 + res_sd.x * d_sd
44 print(f"SD/BFGS/FR: x1 = {x1_sd}, f(x1) = {f(x1_sd)}")
45
46 # --- 2. Newton ---
47 d_newton = np.linalg.solve(H0, -g0)
48 # Exact Line Search
49 res_newton = minimize_scalar(lambda alpha: f(x0 + alpha * d_newton))
50 x1_newton = x0 + res_newton.x * d_newton
51 print(f"Newton: x1 = {x1_newton}, f(x1) = {f(x1_newton)}")

```

Κώδικας 5: Υλοποίηση SD και FR με Armijo (Άσκηση 3c)

```

1 import numpy as np
2 import time
3
4 #
5 def f1(x): return x[0] - x[1] * (2 - x[1] * (5 - x[1])) - 13

```

```

6 def f2(x): return x[0] - x[1] * (14 - x[1] * (1 + x[1])) - 29
7 def f(x): return f1(x)**2 + f2(x)**2
8
9 def gradient_f(x):
10     df1_dx1 = 1.0
11     df1_dx2 = -2 + 10*x[1] - 3*x[1]**2
12     df2_dx1 = 1.0
13     df2_dx2 = -14 + 2*x[1] + 3*x[1]**2
14     g1 = 2 * f1(x) * df1_dx1 + 2 * f2(x) * df2_dx1
15     g2 = 2 * f1(x) * df1_dx2 + 2 * f2(x) * df2_dx2
16     return np.array([g1, g2])
17
18 # Line Search (Armijo)
19 def line_search_armijo(x, d, g, f_val):
20     alpha = 1.0
21     c1 = 1e-4
22     beta = 0.5
23     while True:
24         x_new = x + alpha * d
25         if f(x_new) <= f_val + c1 * alpha * np.dot(g, d):
26             return alpha, x_new, f(x_new)
27         alpha *= beta
28         if alpha < 1e-16: return alpha, x, f_val # Safety break
29
30 # Steepest Descent
31 def steepest_descent(x0, tol=1e-8, max_iter=5000):
32     x = np.array(x0)
33     start = time.time()
34     for k in range(max_iter):
35         g = gradient_f(x)
36         if np.linalg.norm(g) < tol: break
37         alpha, x, f_val = line_search_armijo(x, -g, g, f(x))
38     return k, time.time() - start
39
40 # Fletcher-Reeves
41 def fletcher_reeves(x0, tol=1e-8, max_iter=5000):
42     x = np.array(x0)
43     g = gradient_f(x)
44     d = -g
45     start = time.time()
46     for k in range(max_iter):
47         if np.linalg.norm(g) < tol: break
48         alpha, x_new, f_new = line_search_armijo(x, d, g, f(x))
49         g_new = gradient_f(x_new)
50         beta = np.dot(g_new, g_new) / (np.dot(g, g) + 1e-10)
51         d = -g_new + beta * d
52         x, g = x_new, g_new
53     return k, time.time() - start

```